CSCI 1851: Homework 2 Writeup

1) Cancer Genomics Dataset

The cancer genomics dataset contains gene expression features (ex. gene_18, gene_21, etc) and a separate label csv with tumor types. The most important preprocessing was combining these, which was done for us in hw2_loader. There were no missing values or duplicated features, and the data had already been feature-selected and cleaned. I could've done more feature selection for a faster runtime but scores were looking alright at a fast-ish . It was high dimensionality, with 801 samples and 5479 features. Classes were moderately unbalanced–BRCA: 300, KIRC: 146, LUAD: 141, PRAD: 136, COAD: 78, so a stratified train/test split was implemented for classification, and a Macro F1 or Macro ROC-AUC will be more helpful than accuracy. Feature scale appeared pretty continuous, ranging from 0.0 to ~20.

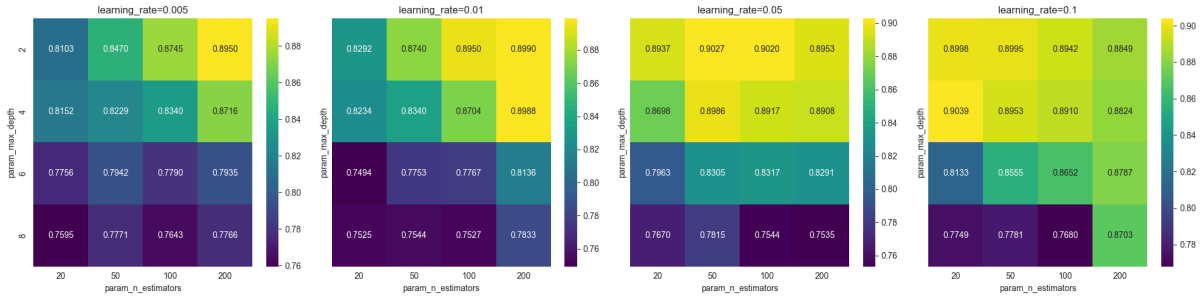
2) Original Model Performance (Heart Disease + Cancer Genomics)

The cancer dataset was much easier to classify: as written on Ed, the dataset had already been heavily preprocessed, which can make the task highly separable. The dominant type error in the cancer dataset was false negatives, as signaled by the slightly lower recall versus precision, but these error appear very minimal given the results of cross-validation:

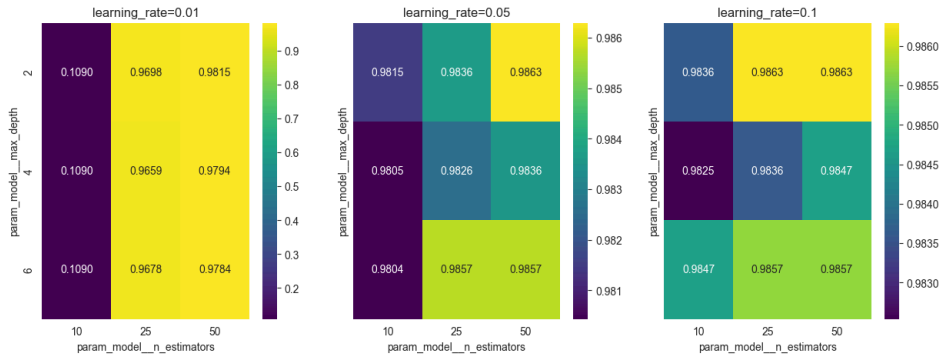
	Heart Dataset	Cancer Dataset
accuracy	0.8111	0.9938
precision	0.8044	0.9968
recall	0.8661	0.9931
f1	0.8334	0.9949
roc_auc	0.8925	0.9999

3) Hyperparameter Tuning (Gradient Boosting)

I tested max_depth: [2, 4, 6, 8], n_estimators': [20, 50, 100, 200], and 'learning_rate': [0.005, 0.01, 0.05, 0.1] for the heart, and max_depth: [2, 4, 6], n_estimators: [10, 25, 50], and learning_rate: [0.01, 0.05, 0.1]. I considered also testing other hyperparameters like subsample, min_samples_split, & min_samples_leaf, but due to long runtimes I kept it simple.



The heart plot performed best (90.39%) with a learning_rate of 0.1, max_depth of 4, and 20 estimators. It was interesting to see that a shallower tree generally worked better, likely because the dataset is relatively small and deeper trees may overfit the training data. The higher learning rate also worked well in combination with a small number of estimators, allowing each tree to contribute more strongly to the final model.

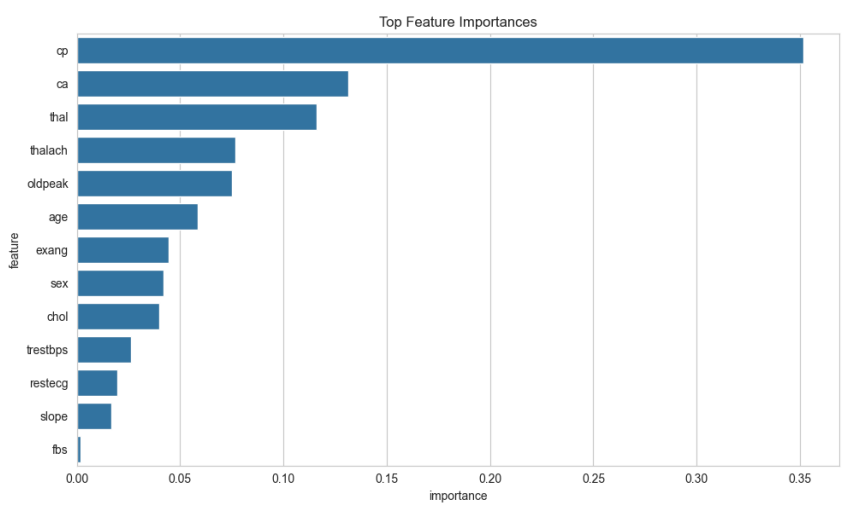


* Keep in mind the changing color scales for these.

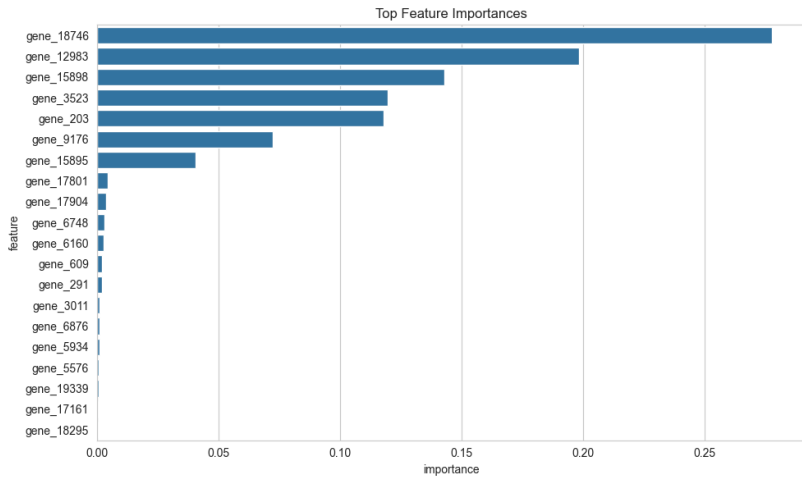
For the cancer plot, the ideal parameters were learning_rate of 0.05, max_depth of 2, 50 estimators, and a score of 98.63%-- a good balance between more estimators, a small-ish learning rate, and a short depth (which might be important in boosting because weak learners won't overfit and introduce too much variance). As a general rule, the more estimators, the better the model performed, but that also might be limited because we only tested up to 50. In contrast, a larger learning rate with only 10 estimators performed pretty horribly, which makes sense because you're taking large steps but stopping almost immediately.

4) Feature Importance and Interpretation

For the heart plot, ca and cp were the most predictive, which aligns with my homework 1 results. However, sex was a much more significant predictive factor in HW1, and here it was barely a factor. This could be due to how linear regression relies on linear relationships, but decision trees can capture nonlinear relationships and feature outcomes. Because tree-based models split on features that provide the largest reduction in impurity, they may prioritize variables that produce strong decision boundaries or interactions with other variables. If sex mainly contributes through a weaker linear effect or through relationships already captured by other variables, its relative importance may appear smaller in the gradient boosting model.



For the cancer plot, 7 features have significant importance on the output, with gene_18746 leading at around 28%.



5) Comparison to HW1 (Cancer only)

I selected the top 7 features due to gradient boosting, and trained a logistic regression model on the full set and reduced set. The reduced feature set slightly improved performance on everything but accuracy, which was tied:

	accuracy	precision	recall	f1	roc_auc
full	0.987578	0.980357	0.980357	0.980357	0.999720
Top 7	0.987578	0.993548	0.985714	0.989314	0.999827

It is likely that the extra features were adding noise rather than meaningful signals, and so limiting it to the most impactful features allowed for slightly better performance than on the full dataset.

6) Discussion

Gradient boosting assumes that training data is representative of the true distribution, which is a meaningful constraint here given that the heart disease dataset is relatively small and may not generalize across different demographic or clinical populations. Even on the cancer genomics dataset, where performance was strong, results were evaluated on a fixed split and may not hold across different patient cohorts or sequencings. Overfitting was a visible concern in hyperparameter tuning, requiring a balance of depth, estimators, and learning rate; the heart dataset performed best with max_depth=4 and only 20 estimators at a higher learning rate of 0.1, suggesting that with a smaller dataset, shallower trees and fewer boosting rounds are enough to avoid overfitting while still letting each tree contribute strongly. In turn, the cancer dataset favored max_depth=2 with 50 estimators at a more conservative learning rate of 0.05, where keeping trees shallow likely prevented individual weak learners from introducing too much variance into the ensemble. Data quality introduced some uncertainty as well, particularly for the heart disease dataset where duplicate records required removal. Medical data is also inherently messy, and subject to collection bias surrounding who can access medical care. The top-7 feature selection was driven by GBM feature importances, which measure splitting utility rather than true causal relevance, so these importances should be interpreted with some caution. Finally, the compute size and resulting long runtime of the cancer dataset didn't allow us to explore as many hyperparameters as would be ideal, and so the reported hyperparameters might not be the actual optimal ones.